

PYTHON FOUNDATIONS

Python Basics Quick Reference Guide

An abbreviated guide for Python Basics Comprehension

How to use this guide

This guide covers key topics to help with Python Basics comprehension. Read each section, try the “Quick check” prompt before moving on, and pay close attention to the yellow ⚠ boxes — they call out the mistakes students make most often.

Sections 1–6 cover Python as a language: its features, history, community standards, and where it’s used. Sections 7–17 cover the building blocks you’ll be tested on with code: variables, strings, arrays, functions, and exception handling.

1. Key Features of Python

Python is a high-level, general-purpose programming language. “High-level” means it reads much closer to plain English than to the 1s and 0s a computer actually runs — Python handles the low-level details for you.

Core features to know

- **Interpreted:** Python code runs line-by-line through an interpreter, rather than being compiled into a standalone executable ahead of time. This makes testing and debugging fast.
- **Dynamically typed:** you don't have to declare a variable's type. Python figures it out — and can change it — at runtime (score = 90, then later score = "A" is legal, though not always a good idea).
- **Automatic memory management:** Python allocates and frees memory for you through garbage collection. You never manually manage pointers, unlike lower-level languages such as C.
- **Extensive standard library:** Python ships with built-in modules for math, file handling, dates, networking, and more — you don't have to build everything from scratch.
- **Readable syntax:** Python uses indentation (not curly braces) to define blocks of code, which forces a consistent, readable visual structure.
- **Cross-platform:** the same Python code generally runs on Windows, macOS, and Linux without changes.

⚠ Worth remembering

It's easy to assume Python requires manual memory management, the way lower-level languages like C do. It doesn't — automatic garbage collection is one of Python's defining features, freeing you from tracking and releasing memory by hand.

Quick check: Could you explain, in one sentence, why Python is called a “dynamically typed” language?

2. Industry Leaders Using Python

Python isn't just a teaching language — it's used at massive scale in production systems across many industries.

- Google has used Python extensively since its early days, for everything from internal tooling and automation to parts of its production infrastructure.
- Netflix and Spotify rely on Python for data pipelines, automation, and backend services.
- Instagram has run large parts of its backend on Python (via the Django framework) at a massive scale.
- Beyond “tech companies,” Python is heavily used in finance, science, and research for data analysis, automation, and rapid prototyping.

The common thread: Python's combination of readability, a huge library ecosystem, and fast development speed makes it attractive for organizations that need to build and iterate quickly.

Quick check: Name one company discussed in class that uses Python, and one reason why it might have chosen Python.

3. History of Python

- Python was created by Guido van Rossum, who began developing it in the late 1980s as a successor to a language called ABC.
- The first official release, Python 0.9.0, came out in 1991.

- Python 2.0 was released in 2000, adding features like list comprehensions and garbage collection improvements.
- Python 3.0, released in 2008, was a significant, partly backward-incompatible redesign that cleaned up inconsistencies in the language. Python 3.x is the version used today.
- The name “Python” comes from the British comedy series Monty Python's Flying Circus — not the snake — reflecting van Rossum's goal of making programming a bit more fun.

⚠ Worth remembering

It's easy to mix up language creators when several languages come up in the same review session. Anchoring on “Guido van Rossum = Python” as its own fact, separate from other creator/language pairs, tends to stick better than memorizing a list.

Quick check: Who created Python, and roughly what year did it first release?

4. Python Enhancement Proposals (PEPs)

A PEP (Python Enhancement Proposal) is a design document used by the Python community to propose, describe, and discuss new features, processes, or standards for the language itself.

Why PEPs matter

- PEPs are how Python evolves in an organized, documented way — instead of changes happening ad hoc, they're proposed, debated, and formally recorded.
- Some PEPs are famous in their own right. PEP 8 defines the style guide for writing Python code (covered next). PEP 20, “The Zen of Python,” lists guiding principles like “Readability counts.”
- PEPs govern the Python language and its core tooling — they are not the same thing as third-party packages on PyPI, and they are not an IDE or development tool.

Quick check: In your own words, what problem do PEPs solve for a language used by millions of developers?

5. Importance of PEP 8

PEP 8 is the official style guide for Python code. It doesn't change how code runs — it standardizes how code looks, so that Python written by different people stays readable and consistent.

What PEP 8 covers

- Naming conventions, such as `snake_case` for variables and functions (`total_score`, not `TotalScore`).
- Whitespace and indentation rules, such as using 4 spaces per indentation level.
- Line length guidance (generally, keep lines under 79–99 characters).
- Conventions for comments, blank lines, and import statement ordering.

⚠ Worth remembering

PEP 8 is about readability and consistency, not correctness. Code that ignores PEP 8 conventions can still run correctly — it's just harder for other people (including future you) to read and maintain.

Quick check: Why might following PEP 8 matter more on a team project than in a solo script you'll never share?

6. Python for Data Analysis & Engineering

Python is one of the most widely used languages in data analysis, data engineering, and data science. Key reasons include:

- A mature library ecosystem: pandas (data manipulation), NumPy (numerical computing), and Matplotlib/Seaborn (visualization) are industry-standard tools built specifically for working with data.
- Strong integration with databases and APIs — Python connects easily to SQL databases, cloud services, and web APIs to pull in real data.
- Readable syntax lowers the barrier for analysts and scientists who aren't full-time software engineers.
- A large, active community means answers, tutorials, and pre-built solutions are easy to find.

Quick check: Name two Python libraries commonly used in data analysis, and what each one is generally used for.

7. Python Data Types

Every value in Python has a data type — a classification that determines what the value represents and what operations can be performed on it. You've already met several data types in this guide (str, int, float, list); this section pulls them all together in one place.

Python organizes its built-in data types into a handful of families. The `type()` function tells you the type of any value:

```
print(type(16))           # <class 'int'>
print(type("hello"))      # <class 'str'>
print(type([1, 2, 3]))    # <class 'list'>
```

The data type families

- Numeric — whole numbers, decimals, and (rarely) complex numbers: int, float, complex.
- Boolean — the two truth values, True and False: bool.
- Text — strings of characters: str.
- Sequence — ordered collections accessed by position: list, tuple, range.
- Mapping — collections of key-value pairs: dict.
- Set — unordered collections of unique values: set, frozenset.
- Binary — raw byte data, used for files and network data: bytes, bytearray.
- None — represents the absence of a value: NoneType, whose only value is None.

⚠ Worth remembering

Whether a type is mutable (changeable after creation) or immutable (locked once created) usually matters more day-to-day than which family it belongs to. list and dict are mutable; str, tuple, and range are not — even though a list and a tuple are both “sequences.”

Data types at a glance

Type	Example	Mutable?	What it's for
int	age = 16	No	Whole numbers, positive or negative, with no decimal point.
float	gpa = 3.8	No	Numbers with a decimal point, used for measurements and precise math.
complex	z = 2 + 3j	No	Numbers with a real and imaginary part; used mainly in specialized math and engineering.
bool	is_passing = True	No	Represents True or False; used for conditions and comparisons.
str	name = "Maria"	No	A sequence of characters used to represent text.
list	grades = [90, 85, 78]	Yes	An ordered, changeable collection that can hold mixed data types.
tuple	point = (3, 4)	No	An ordered, unchangeable collection, often used for fixed groups of values.
range	range(5)	No	An immutable sequence of numbers, most often used to control loops.
dict	student = {"name": "Sam"}	Yes	A collection of key-value pairs, looked up by key rather than position.
set	colors = {"red", "blue"}	Yes	An unordered collection of unique values; automatically removes duplicates.
frozenset	frozenset({"a", "b"})	No	An immutable version of a set.
bytes	data = b"hello"	No	An immutable sequence of raw byte values, used for binary data.
bytearray	buffer = bytearray(5)	Yes	A mutable sequence of raw byte values.
NoneType	result = None	No	Represents the absence of a value — Python's version of “nothing here.”

⚠ Worth remembering

bool is technically a subtype of int in Python — True behaves like 1 and False behaves like 0 in arithmetic (True + True evaluates to 2). It's still worth treating Boolean as its own family conceptually, since it's used for conditions and logic, not quantities.

Quick check: Pick any two data types from the table above with the same 'Mutable?' value. What do they have in common, and how do they differ?

8. What Variables Are

A variable is a named reference used to store and refer to data that a program creates, uses, or modifies. Think of a variable as a labeled box: the label (the variable name) lets you find and use whatever value is stored inside.

```
student_name = "Maria"
age = 16
gpa = 3.8

print(student_name, age, gpa)
```

Here, `student_name`, `age`, and `gpa` are all variables. The `=` sign is the assignment operator — it stores the value on the right into the variable name on the left. Once assigned, a variable's value can be read, used in calculations, or reassigned later in the program.

⚠ Worth remembering

A variable isn't a value that's locked in forever. Its defining feature is that the value it points to can change as the program runs — that's what makes it “variable” in the first place.

Quick check: What is the purpose of a variable, and how is that different from a fixed, unchanging value?

9. Rules for Naming Variables in Python

Python enforces a small set of hard rules for variable names. Break these and your code won't run:

- Names can contain letters, numbers, and underscores (`_`) only — no spaces, hyphens, or other symbols.
- Names cannot start with a number. `student1` is valid; `1student` is not.
- Names cannot be a Python reserved keyword, such as `if`, `for`, `class`, or `True`.
- Names are case-sensitive: `score`, `Score`, and `SCORE` are three different variables.

```
# Valid variable names
total_points = 82
_temp_grade = 91
finalScore = 76

# Invalid — starts with a digit, causes a SyntaxError
3rd_attempt = "Sam"
```

Rules vs. style conventions

Beyond the hard rules above, PEP 8 recommends (but doesn't require) using `snake_case` for variable names — all lowercase, words separated by underscores, like `total_score` rather than `totalScore`. `camelCase` names like `totalScore` are legal Python — they will run without error — they're just not the idiomatic style.

⚠ Two different kinds of “wrong”

It helps to separate two ideas: rules Python enforces, and conventions PEP 8 recommends. Breaking one of Python's actual naming rules stops your code from running at all. Going against a PEP 8 convention (like using camelCase instead of snake_case) doesn't — the code still runs, it's just not the preferred style.

Quick check: Of the four example names above, which one won't run — and which naming rule does it break?

10. Strings and How They Differ from Other Data Types

A string is a sequence of characters — letters, numbers, symbols, or spaces — enclosed in quotes (either single ' or double " marks). Strings are used to represent text.

```
first_name = "Jordan"
greeting = 'Hello, World!'
zip_code = "19104"    # a string, even though it's all digits
```

How strings differ from numeric types

- Numeric types (int, float) represent numbers you can do math with directly (age + 1). Strings represent text — even a string made of digits, like "19104", cannot be used in arithmetic without first converting it to a number.
- Strings are immutable: once created, a string's contents cannot be changed in place. Any operation that appears to “modify” a string (like .upper() or string concatenation) actually creates and returns a brand-new string.
- Strings support text-specific operations that numbers don't have, like indexing individual characters, slicing substrings, and methods such as .lower(), .strip(), and .replace().

⚠ Worth remembering

It's tempting to think you can change one character of a string directly, the way you might update one item in a list. You can't — a string's characters can't be reassigned in place. Any “change” you see is really a new string being created and stored.

Quick check: Why can't "19104" + 1 be evaluated the same way age + 1 can?

11. Comparing Strings with Comparison Operators

Python's comparison operators (==, !=, <, >, <=, >=) work on strings just like they do on numbers, but string comparisons are case-sensitive and compare characters in order (this is often called “lexicographic” comparison).

```
city1 = "Boston"
city2 = "boston"

print(city1 == city2)           # False — case-sensitive
```

```
print(city1.lower() == city2.lower())    # True — both normalized to lowercase
print("apple" < "banana")               # True — 'a' comes before 'b'
```

Because `==` checks for exact character-by-character equality, "Boston" and "boston" are NOT equal — the capital B makes them different strings. To compare text while ignoring case, normalize both sides first, typically with `.lower()` or `.upper()`.

⚠ Worth remembering

Don't assume Python treats two strings that only differ in capitalization as equal. Unless you explicitly normalize case with `.lower()` or `.upper()`, string comparisons are case-sensitive by default.

Quick check: Predict the output of `"Cat" == "cat"` and explain why.

12. What an Array Is

An array is a data structure that stores a fixed-type, ordered collection of elements. Arrays are typically used when you need to store many values of the same type — often numbers — efficiently, and access them by position (index).

- Ordered: elements have a defined position, starting at index 0.
- Fixed-type: every element in a true array must be the same declared type (for example, all integers, or all floating-point numbers).
- Efficient storage: because every element is the same type and size, arrays can be stored compactly in memory — which matters for large datasets or performance-sensitive code.

Quick check: In your own words, what problem does an array solve that a single variable can't?

13. Distinguishing Between Lists and Arrays

Python's built-in list is the data structure you'll use most often to store collections of values. Python also provides a separate array module for true, single-type arrays. They look similar but behave differently.

Python list	array (array module)
Can store mixed data types in one list, e.g. <code>[1, "two", 3.0]</code>	Requires every element to be the same declared type (set by a typecode)
Built into the language — no import needed	Requires <code>import array</code>
Created with square brackets: <code>my_list = [1, 2, 3]</code>	Created with a typecode: <code>arr.array('i', [1, 2, 3])</code>
More flexible; used far more often in everyday Python	More memory-efficient for large, uniform numeric datasets
Grows/shrinks easily with <code>.append()</code> , <code>.insert()</code> , <code>.remove()</code>	Supports similar methods, but all elements must match the typecode

```
# A Python list — mixed types are fine
mixed_list = [1, "two", 3.0, True]
```



```
# An array — every element must be the declared type ('i' = integer)
import array as arr
numbers = arr.array('i', [1, 2, 3])
```

⚠ Worth remembering

Curly braces {} in Python create a set or a dictionary — never an array. There's no special array syntax built into the language the way [] creates a list; true arrays always come from the array module (or a library like NumPy).

Quick check: What is the single biggest difference between a Python list and an array module array?

14. Basic Operations on Arrays: Insertion, Deletion, Traversal

Three operations come up constantly when working with arrays (and lists, which share similar methods):

- Insertion — adding a new element. `.append(x)` adds `x` to the end; `.insert(index, x)` adds `x` at a specific position, shifting later elements over.
- Deletion — removing an element. `del my_array[index]` removes the element at that position; `.remove(value)` removes the first element matching that value.
- Traversal — visiting every element in order, most often with a `for` loop.

```
import array as arr

grades = arr.array('i', [72, 85, 91])
grades.append(88)      # insertion at the end -> [72, 85, 91, 88]
grades.insert(1, 79)   # insertion at index 1 -> [72, 79, 85, 91, 88]
del grades[0]          # deletion by index    -> [79, 85, 91, 88]

for grade in grades:   # traversal
    print(grade)
```

Tracing through the example above step by step: the array starts as `[72, 85, 91]`. `append(88)` adds to the end. `insert(1, 79)` inserts 79 at index 1, shifting everything after it one position to the right. `del grades[0]` removes whatever is currently at index 0 (72) — not a specific value, but whatever occupies that position at the time. The final traversal prints each remaining element in its new order.

⚠ Worth remembering

After you insert or delete an element, every element after that position shifts to a new index. The most common tracing mistake is forgetting that shift and using the original index positions instead of the current ones. Walk through operations one line at a time, redrawing the array after each step.

Quick check: If you insert an element at index 0 of a 3-element array, what index does the original first element move to?

15. Using Python's Built-In array Module

To create a true, single-type array in Python, import the built-in array module. Every array is created with a typecode that declares what kind of value it holds — this is what enforces the “fixed-type” rule.

```
import array as arr

readings = arr.array('d', [12.5, 14.2, 13.8])  # 'd' = double-precision float
print(readings.typecode)  # 'd'
print(len(readings))      # 3
print(readings[0])        # 12.5
```

Common typecodes

- 'i' — signed integer
- 'f' — floating-point number (single precision)
- 'd' — floating-point number (double precision)
- 'u' — Unicode character

Trying to insert a value that doesn't match the array's typecode (for example, appending a string to an 'i' array) raises a `TypeError` — this enforcement is exactly what makes array module arrays different from Python lists.

Quick check: If you create an array with `arr.array('i', [1, 2, 3])`, what happens if you try to append 4.5 to it?

16. What a Function Is

A function is a reusable, named block of code that performs a task. Instead of writing the same steps over and over, you define them once, give them a name, and run (“call”) that name whenever you need the task done again.

Defining vs. calling a function

```
def convert_to_percent(correct, total):  # definition
    ratio = correct / total
    return ratio * 100

grade = convert_to_percent(18, 20)      # call
print(grade)
```

- `def` starts a function definition, followed by the function's name and a set of parentheses.
- Parameters (`correct` and `total` above) are named placeholders listed in the definition — they describe what input the function expects.
- Arguments (`18` and `20` above) are the actual values supplied when the function is called — they fill in the parameters, in order.
- `return` sends a value back to whatever called the function. A function without a `return` statement still runs, but implicitly returns `None`.
- Defining a function doesn't run its code — the code only executes when the function is called.

⚠ Worth remembering

Parameters and arguments are often used interchangeably in casual conversation, but they refer to two different moments: a parameter is the placeholder named in the function's definition, while an argument is the actual value passed in when the function is called.

Why functions matter

- They avoid repeating the same code in multiple places — write the logic once, reuse it anywhere.
- They make programs easier to read, test, and debug, since each function handles one clear task.
- They let you break a large problem into smaller, named pieces.

Quick check: In the example above, which values are the arguments, and which names are the parameters?

17. Exception Handling

An exception is a runtime error — a problem that occurs while a program is running, as opposed to a syntax error, which prevents the program from running at all. Common examples include dividing by zero, accessing a list index that doesn't exist, or trying to open a file that isn't there.

The try/except structure

```
def get_item(items, position):
    try:
        return items[position]
    except IndexError:
        return "That position doesn't exist"

colors = ["red", "green", "blue"]
print(get_item(colors, 1))      # a normal, valid position
print(get_item(colors, 9))      # a position that doesn't exist
```

- Code that might fail goes inside the try block.
- If a matching error occurs, Python immediately jumps to the except block instead of crashing the program.
- `except IndexError` catches only that specific error type — other kinds of errors would not be caught here and would still stop the program.
- If no error occurs in the try block, the except block is simply skipped.

⚠ Worth remembering

A syntax error means Python can't even start running your code — often a typo or a missing colon. An exception happens after the code starts running, when something goes wrong during execution. try/except only handles the second kind; it can't fix a syntax error.

A few common built-in exceptions

- `ZeroDivisionError` — dividing a number by zero.
- `IndexError` — accessing a list or array position that doesn't exist.
- `KeyError` — accessing a dictionary key that doesn't exist.
- `ValueError` — a function receives a value of the right type but an invalid value (e.g., `int("abc")`).
- `TypeError` — an operation is used on a data type that doesn't support it (e.g., `"5" + 5`).

Handling exceptions deliberately — rather than letting a program crash — lets you show a helpful message, use a fallback value, or log the problem and keep going. That's why exception handling matters for writing programs that behave predictably even when something unexpected happens.

Quick check: *Why does `get_item(colors, 9)` not crash the program above, even though index 9 doesn't exist in the list?*

★. Key Terms Glossary

Term	Definition
Interpreted language	A language whose code is run line-by-line by an interpreter, rather than compiled ahead of time.
Dynamically typed	A variable's type is determined automatically at runtime and can change.
PEP	A Python Enhancement Proposal — a design document proposing changes or standards for the language.
PEP 8	The official Python style guide covering naming, spacing, and code layout conventions.
Variable	A named reference used to store and access data that a program can read or modify.
Data type	A classification of a value that determines what it represents and what operations can be performed on it (e.g., int, str, list).
Expression	A piece of code that evaluates to a value, such as <code>3 + 4</code> or <code>len(name)</code> .
Statement	A complete instruction that performs an action, such as an assignment or a function call; a statement may contain one or more expressions.
String	An immutable sequence of characters used to represent text.
Immutable	Cannot be changed in place after creation; any “change” produces a new object.
Array	An ordered, fixed-type collection of elements, often used for efficient numeric storage.
List	Python's built-in, flexible collection type that can hold mixed data types.
Typecode	A single character (e.g., 'i', 'd') that declares the element type of an array module array.
Traversal	Visiting every element of a collection in order, typically with a loop.
Function	A reusable, named block of code that performs a task, optionally accepting inputs and returning a value.
Parameter	A named placeholder listed in a function's definition, describing what input the function expects.

Argument	The actual value supplied to a function when it is called, filling in a parameter.
Return value	The value a function sends back to the code that called it, via a return statement.
Syntax error	An error that prevents code from running at all, typically caused by incorrect structure (e.g., a typo or missing colon).
Exception (runtime error)	An error that occurs while a program is executing, after it has already started running successfully.
try/except block	A structure used to attempt code that might fail (try) and define how to respond if a specific error occurs (except), instead of crashing the program.

Self-check

You should be able to, without looking back at this guide:

- List at least three core features of Python.
- Explain what a PEP is, and why PEP 8 specifically matters.
- Write a valid variable name and explain why an example name is invalid.
- Predict the output of a string comparison that mixes cases.
- Trace an array through an append, an insert, and a delete.
- Explain the key difference between a list and an array.
- Name at least five Python data types and give an example value for each.
- Explain the difference between a mutable and an immutable data type.
- Identify the parameters, arguments, and return value in a short function.
- Explain the difference between a syntax error and an exception (runtime error).
- Trace a try/except block and predict what it prints when an error does - and doesn't - occur.
- Write a short function that processes a list of numbers and returns a calculated result.